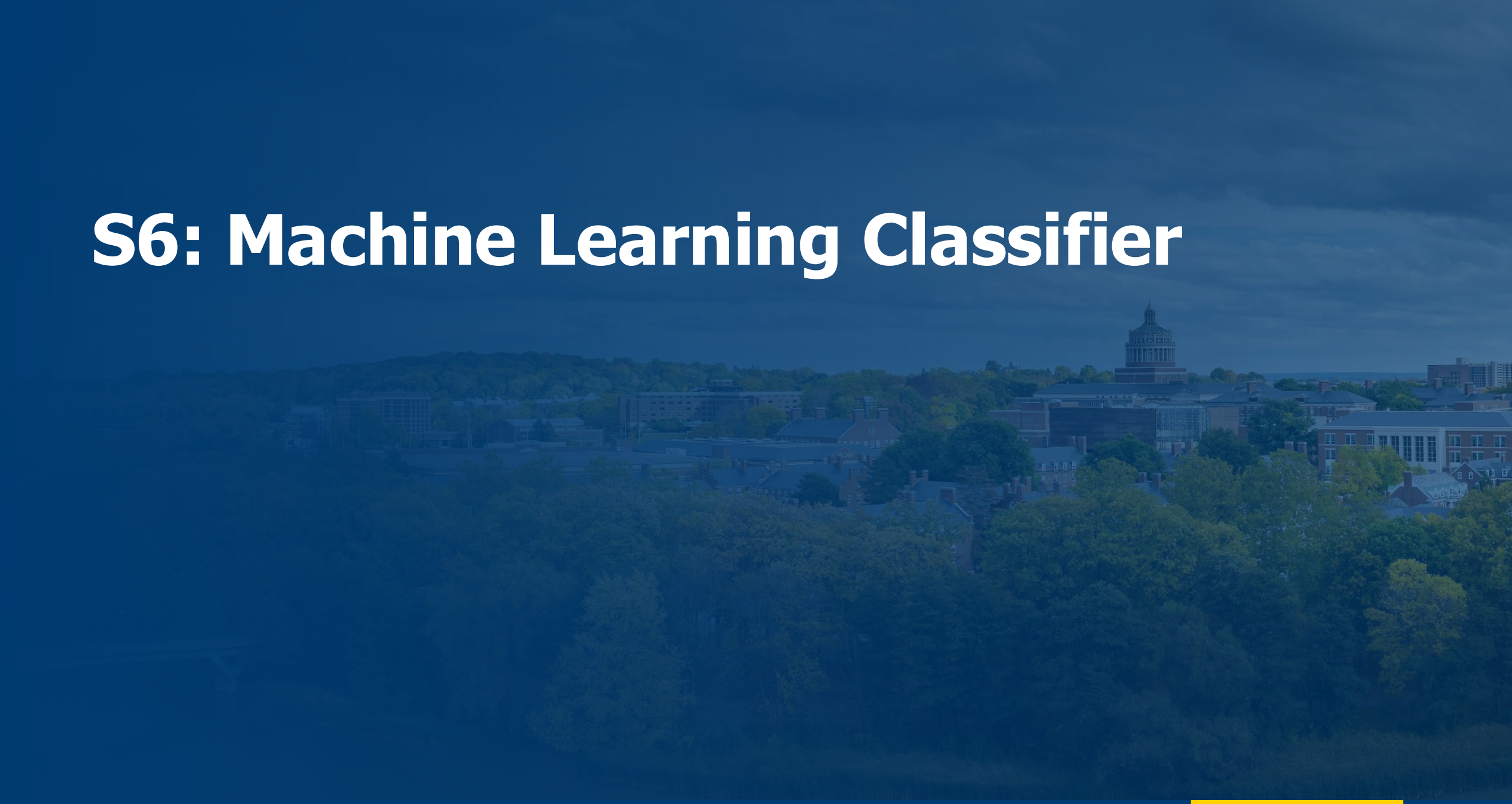
S6: Machine Learning Classifier



Classifiers

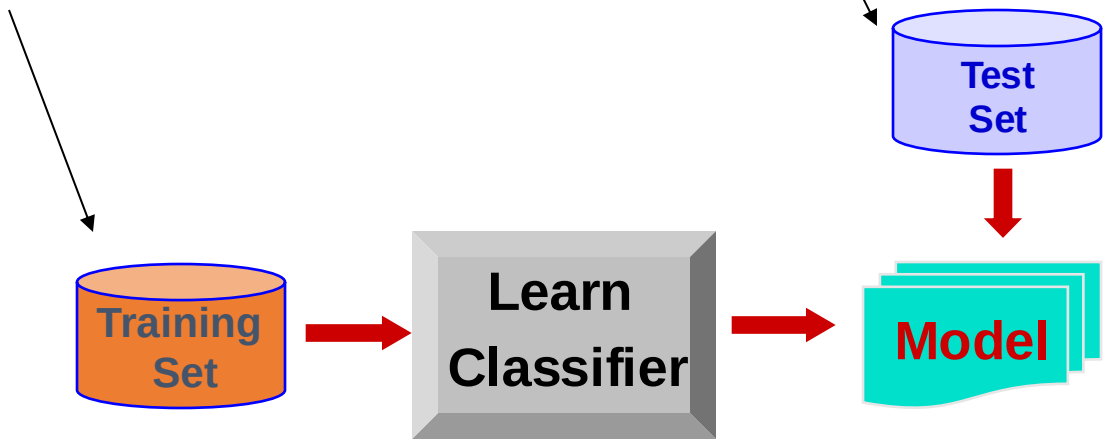
- Human-imposed rule-based
- Weight β -> [0,0,1,-1,0,-0.5]

- Machine learned
- Weight β (parameter)

Train/learn a model

Ex#	text	class: Hooligan
1	An English football fan …	Yes
2	During a game in Italy …	Yes
3	England has been beating France …	Yes
4	Italian football fans were cheering …	No
5	An average USA salesman earns 75K	No
6	The game in London was horrific	Yes
7	Manchester city is likely to win the championship	Yes
8	Rome is taking the lead in the football league	Yes

text	Hooligan
A Danish football fan	?
Turkey is playing vs. France. The Turkish fans …	?



Subjectivity and objectivity: logit

```
## Find 200 sentences with label subj/obj
# NOTE: The sentence is tokenized
from nltk.corpus import subjectivity

n_instances = 200
subj_docs = [(sent, 'subj') for sent in
subjectivity.sents(categories='subj')[:n_instances]]
obj_docs = [(sent, 'obj') for sent in
subjectivity.sents(categories='obj')[:n_instances]]

print(len(subj_docs), len(obj_docs)) #(200, 200)
print(subj_docs[0])
```

Subjectivity and objectivity: logit

```
## split subjective and objective instances to keep a balanced uniform
# class distribution in both train and test sets.
train_subj_docs = subj_docs[:150]
test_subj_docs = subj_docs[150:200]

train_obj_docs = obj_docs[:150]
test_obj_docs = obj_docs[150:200]

# Pool
training_docs = train_subj_docs+train_obj_docs
testing_docs = test_subj_docs+test_obj_docs

# Separate X and Label
training_x = [i[0] for i in training_docs]
testing_x = [i[0] for i in testing_docs]

training_c = [i[1] for i in training_docs]
testing_c = [i[1] for i in testing_docs]
```

Subjectivity and objectivity: logit

```
## TFIDF Vectorization
from sklearn.feature_extraction.text import TfidfVectorizer


# Trick: create a dummy tokenizer
def tk(doc):
    return doc


vec = TfidfVectorizer(analyzer='word',tokenizer=tk,
preprocessor=tk,token_pattern=None,
                      min_df=2, ngram_range=(1,2),
stop_words='english')
vec.fit(training_x)
training_x = vec.transform(training_x)
testing_x = vec.transform(testing_x)
```

Subjectivity and objectivity: logit

```
## Logit
from sklearn.linear_model import LogisticRegression
Logitmodel = LogisticRegression()

# training
Logitmodel.fit(training_x, training_c)
y_pred_logit = Logitmodel.predict(testing_x)

# evaluation
from sklearn.metrics import accuracy_score

acc_logit = accuracy_score(testing_c, y_pred_logit)
print("Logit model Accuracy:: {:.2f}%".format(acc_logit*100))
```

Tuning for improving model *f*(β',[X]) performance

- β' is determined by ? [X], [c], and f(,)
- Sample size of [X] and [c]
  - Minimal value
- Representation [X]
  - N-grams
  - Minimal document frequency
  - Overfitting problem
- The model frameworks *f*(,)

General classification

- In logit, the specification f(,) is s=x β’ <> 0 or cutoff (fixed)
- This specification is restrictive: ‘not good’ vs. ’bad’
- General classification: relax the restriction on the specification f(,)
- Any model f(,) that map [X] to C is valid for classification
- Alter the specification to any classification models and train the models if
  - We have hand-classified training data
  - (No free lunch ▪, but data can be built up and refined by amateurs)

Candidate classification models

- Naïve Bayes
- **Logit Model**

classic, works well (70% accuracy), computationally savvy (CPU)

- Support Vector Machine
- Decision Tree(Forest)
- Neural Networks

advanced, works well (80% accuracy), computationally savvy (CPU)

- Deep Neural Networks
- …

frontier, works very well (90+% accuracy), computationally heavy (GPU)

- Note: using the **same** training set and testing set for comparing the performance of alternative models

Naïve Bayes Model

- The simplest classifier
- Naive Bayes won 1st and 2nd place in KDD-CUP 97 competition out of 16 text classification systems
- A good dependable baseline for text classification (but not the best)!

- **Naïve**: assuming independence between X variables
- **Bayes**: using Bayes updating rule
  - $P(c|X) = \frac{P(c,X)}{P(X)} = P(X|c)P(c)/\sum_c P(c,X)$
    $= P(X|c)P(c)/\sum_c(P(X|c) * P(c))$

Bayes rule: football example

X1: Host/Guest (1/0)	X2: Snow/Not (1/0)	C: Win/Not (1/0)
1	1	0
0	0	0
1	0	1
0	1	1
1	1	1
0	0	0
1	1	?

$$P(win|host, rain)$$
$$= \frac{P(host|win) * P(snow|win) * P(win)}{P(host, snow, win) + P(host, snow, loss)}$$
$$= \frac{\frac{2}{3} * \frac{2}{3} * \frac{1}{2}}{\frac{2}{3} * \frac{2}{3} * \frac{1}{2} + \frac{1}{3} * \frac{1}{3} * \frac{1}{2}} = \frac{4}{5}$$

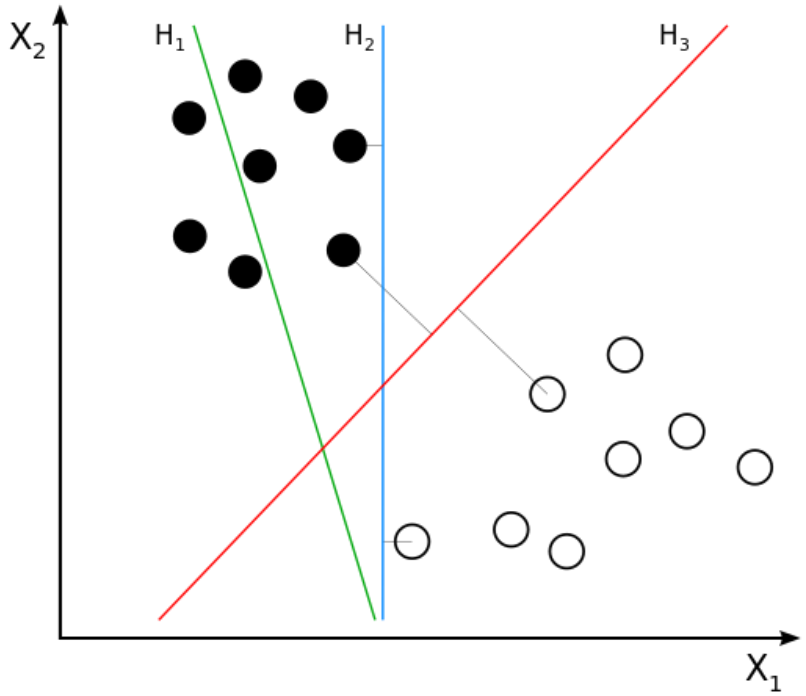
$$P(loss|host, snow)$$
$$= \frac{P(host|loss) * P(snow|loss) * P(loss)}{P(host, snow, win) + P(host, snow, loss)}$$
$$= \frac{\frac{1}{3} * \frac{1}{3} * \frac{1}{2}}{\frac{2}{3} * \frac{2}{3} * \frac{1}{2} + \frac{1}{3} * \frac{1}{3} * \frac{1}{2}} = \frac{1}{5}$$

Python: Naïve Bayes

```
## Naive Bayes
from sklearn.naive_bayes import MultinomialNB
NBmodel = MultinomialNB()

# training
NBmodel.fit(training_x, training_c)
y_pred_NB = NBmodel.predict(testing_x)

# evaluation
acc_NB = accuracy_score(testing_c, y_pred_NB)
print("Naive Bayes model Accuracy:: {:.2f}%".format(acc_NB*100))
```

Support Vector Machine (SVM)

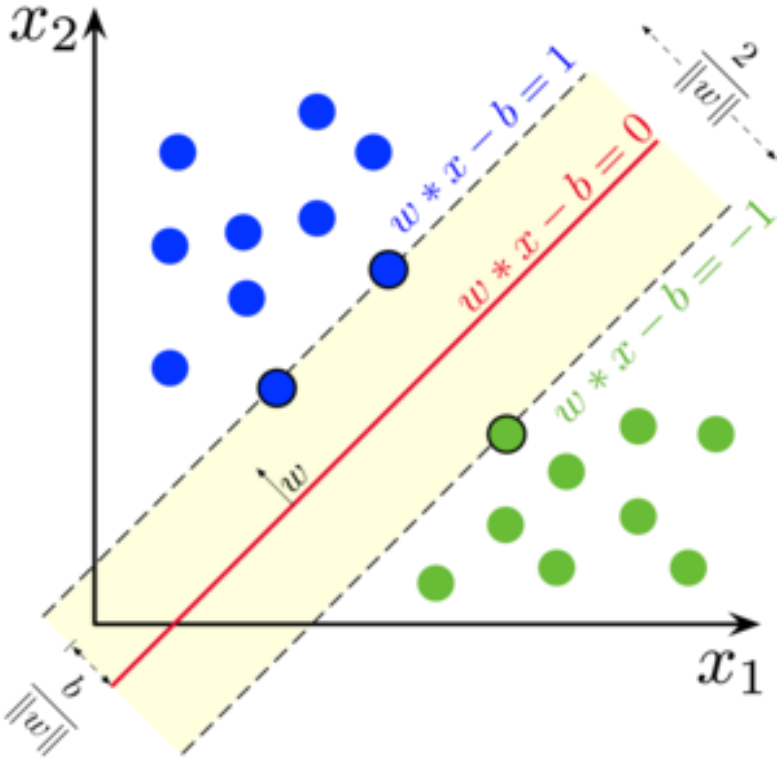
- Find a (set of) hyperplane(s) that
- Separate different classes
- Have the largest separation(margin): the distance from it to the nearest data point on each side is maximized



Math of SVM

- **Support Vector**: the **points** that the **margin** hits.
  - **Boundary 1**: $w * x - b = 1$
    - anything on or above this boundary is of one class, with label **1**
    - $w * x - b \geq 1$ then $y = 1$
  - **Boundary 2**: $w * x - b = -1$
    - anything on or above this boundary is of one class, with label **-1**
    - $w * x - b \leq -1$ then $y = -1$
  - **The hyperplane**: $w * x - b = 0$
  - **The distance** between two boundary: $\frac{2}{||w||}$

$\boldsymbol{w}$ is a scale or a vector?

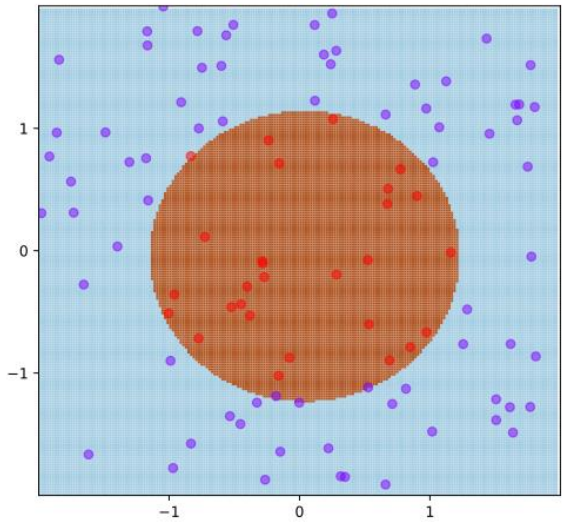


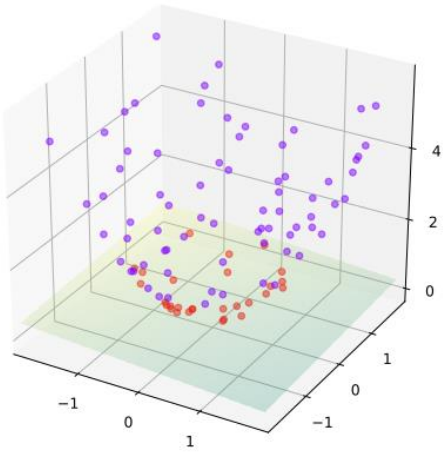
SMV training

- Train the model = find the hyperplane = find $\boldsymbol{w}$ and $\boldsymbol{b}$
- **Maximize** the margin ($\frac{2}{||w||}$) and every dot is classified correctly
- **= Minimize** $||w||$ and subject to that
  - If $w * x - b \geq 1$ then $y = 1$
  - If $w * x - b \leq -1$ then $y = -1$
- **= Minimize** $||w||$ and subject to that
  - $(w * x_i - b)y_i \geq 1$ for $i = 1,2 \ldots, n$

What if not linearly separable?

- **Idea 1:** increase the dimension





- **Idea 2:** relax the condition: minimize the number of points being on the wrong side of the decision boundary
  - **Minimize** $||w||$ and subject to that
  - $(w * x_i - b)y_i \geq 1$ for $i = 1,2 \ldots, n$

- **Minimize**

$$\left[\frac{1}{n}\sum_{i=1}^{n}\max\left(0, 1 - y_i(w \cdot x_i - b)\right)\right] + \lambda\|w\|^2.$$

Python: SVM

```
## SVM
from sklearn.svm import LinearSVC
SVMmodel = LinearSVC()

# training
SVMmodel.fit(training_x, training_c)
y_pred_SVM = SVMmodel.predict(testing_x)

# evaluation
acc_SVM = accuracy_score(testing_c, y_pred_SVM)
print("SVM model Accuracy: {:.2f}%".format(acc_SVM*100))
```

Decision tree

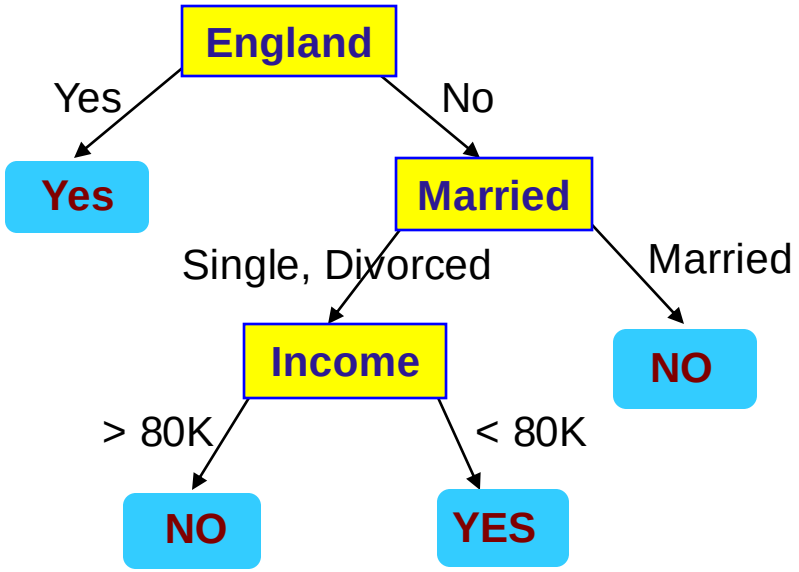
- Decision tree
  - A flow-chart-like tree structure
  - **Internal node** denotes a test on **an attribute X**
  - **Branch** represents an outcome of the test
  - **Leaf nodes** represent **class labels** or **class distribution**
- Classifying an unknown sample
  - Test the attribute of the sample against the decision tree

Learn/train a decision tree

	x1	x2	x3	class
Ex#	Country	Marital Status	Income	Hooligan
1	England	Single	125K	Yes
2	England	Married	100K	Yes
3	England	Single	70K	Yes
4	Italy	Married	40K	No
5	USA	Divorced	95K	No
6	England	Married	60K	Yes
7	England	Divorced	20K	Yes
8	Italy	Single	85K	Yes
9	France	Married	75K	No
10	Denmark	Single	50K	No

Step 1: randomly pick a x

Repeat until a termination criterion is reached
e.g., 90% data within each leaf node has the same label

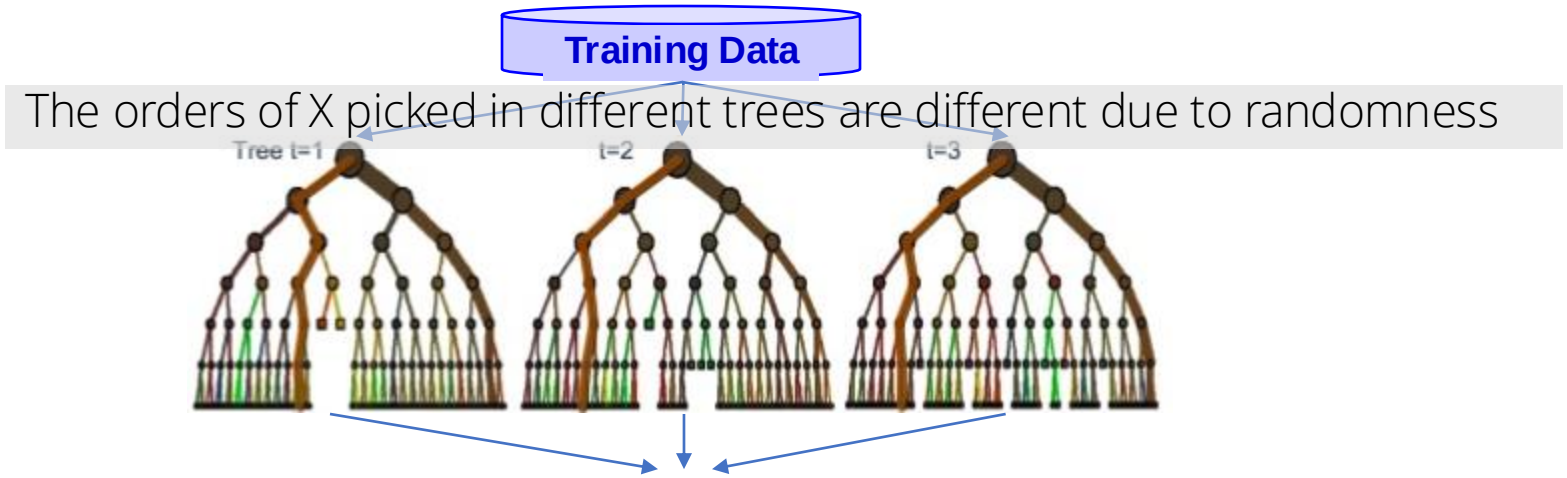


Step 3: within each side, pick another x

Step 2: find threshold for which the classes are as "separate as possible" (on one side, mostly Yes; on other side, mostly No)

Limitation of a single tree, then Decision Forest

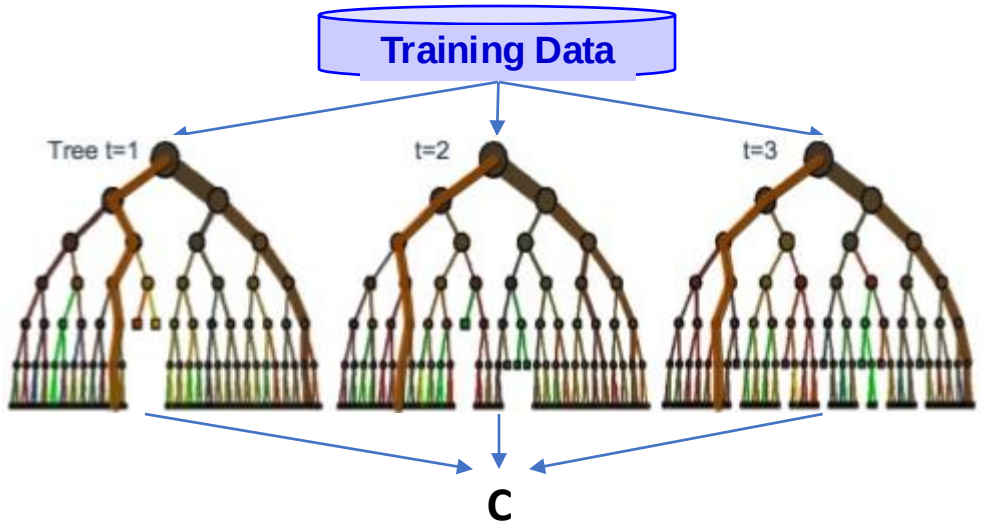
- A decision tree is learned with randomness
  - (e.g., randomly choose x)
- Redo the same learning procedure, we will get **different** decision trees and **different** classification label, good?
  - (e.g., what if we start with **Income** rather than **Country**)
- To stabilize the classification, train and ensemble many trees



- Final prediction = **Majority** of the predicted classes of different trees
- Decision **Tree** -> Decision **Forests** (MANY TREES)

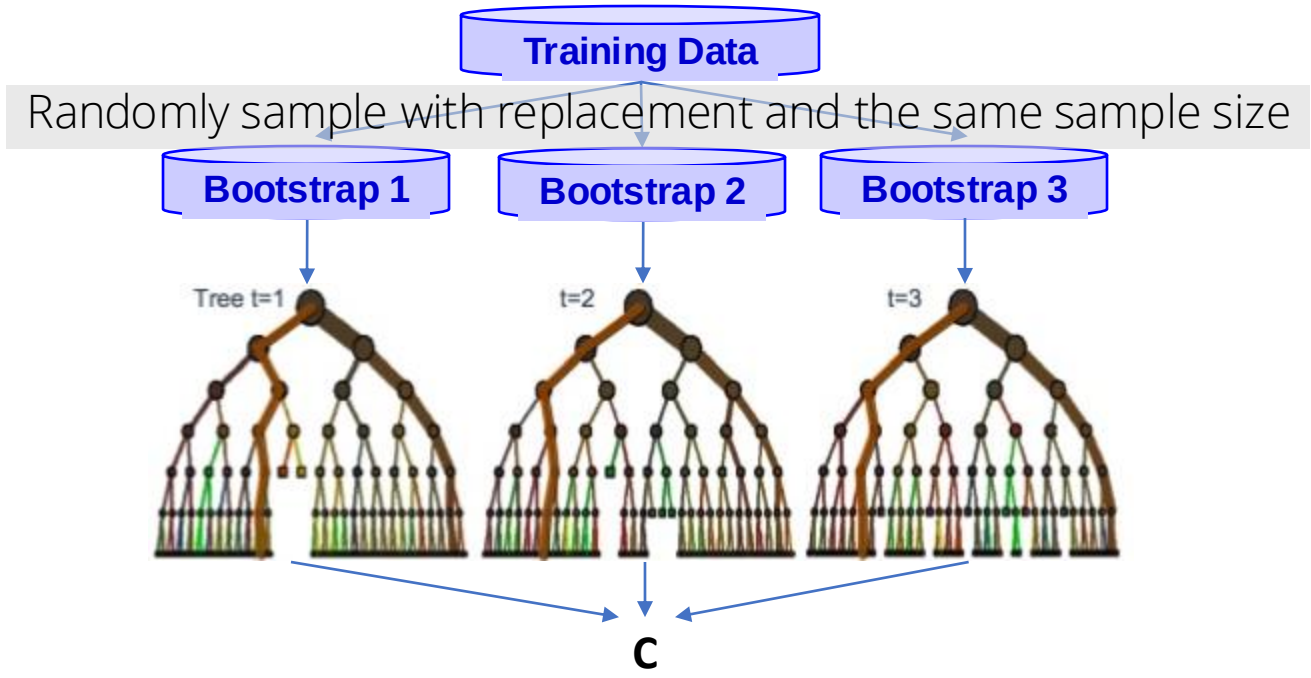
Limitation of Decision Forest, then Random Forest

- **Overfitting**: by feeding all the trees with the same data, the model might just memorize the training data



Random Forest

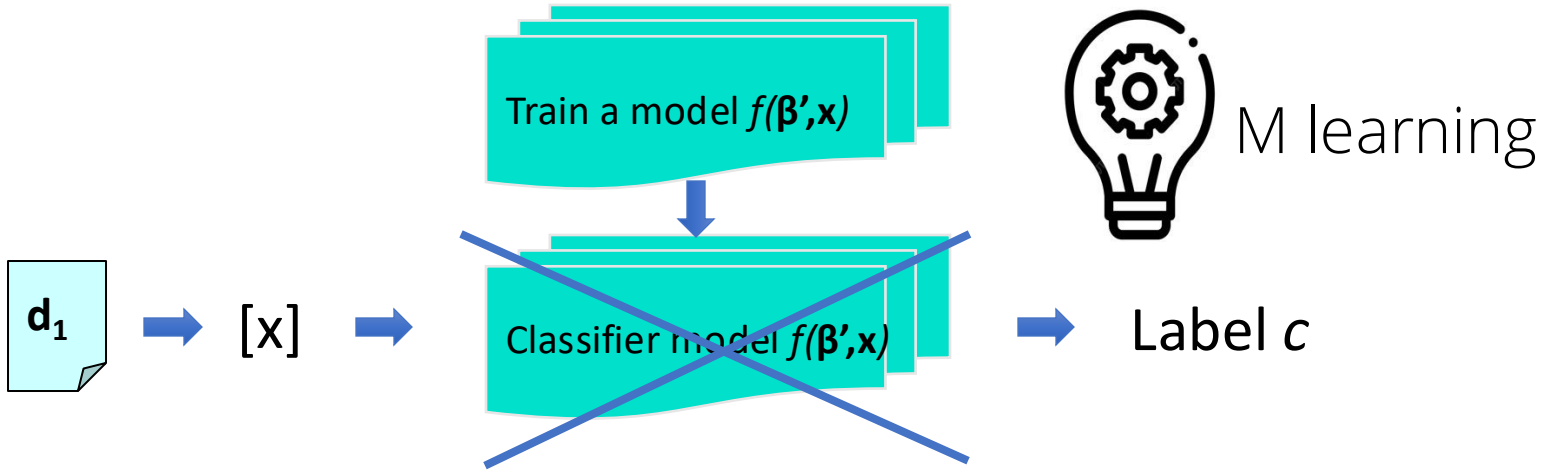
- **Overfitting**: by feeding all the trees with the same data, the model might just memorize the training data
- One more layer of randomness: adding randomness to the training data by **bagging** (**b**ootstrap **agg**regat**ing**)



Python: SVM

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

DTmodel = DecisionTreeClassifier()
RFmodel = RandomForestClassifier(n_estimators=50, max_depth=3, bootstrap=True, random_state=0)
## number of trees and number of layers/depth

# training
DTmodel.fit(training_x, training_c)
y_pred_DT = DTmodel.predict(testing_x)

RFmodel.fit(training_x, training_c)
y_pred_RF = RFmodel.predict(testing_x)

# evaluation
acc_DT = accuracy_score(testing_c, y_pred_DT)
print("Decision Tree Model Accuracy: {:.2f}%".format(acc_DT*100))

acc_RF = accuracy_score(testing_c, y_pred_RF)
print("Random Forest Model Accuracy: {:.2f}%".format(acc_RF*100))
```

Finally, respond



- Now the machine has gained intelligence, a.k.a., the model
- It can respond to any new document d
- Once we recovered β'
  - S = X β'
  - C=1 if S>0 (or P(C=1) > P(C=0)), else C=0

Subjectivity and objectivity

```
new_sent = 'I am a true yellowjacket but I like golden more'
tok_new_sent = nltk.word_tokenize(new_sent)
print(tok_new_sent)

new_x = vec.transform([tok_new_sent])
class_output = Logitmodel.predict(new_x)
print(class_output)
```

Mini Group Project 2b

- There are 1000 reviews for restaurants and films in a collection under the attached csv file. All those reviews are labeled with its category (either restaurant review or movie review). You are required to develop classifiers that could automatically determine whether a future text body is a restaurant review or a movie review. Please follow the steps listed below:
- 1. Build a training dataset with the first 400 restaurant reviews and the first 400 movie reviews (ID = [1:400, 501:900]) and use the rest as a test dataset.
- 2. Following step 1, transform those reviews into a TFIDF matrix, lemmatize all the words, remove the stop-words and punctuations, include uni-gram, and set the minimal document frequency so that the number of columns is just below 200.
- 3. Following step 2, train a Naïve Bayes model, a Logit model, a Random Forest model, (with 50 trees), qnd a SVM model. Calculate the accuracy rate for each of the models, report them and discuss which model performs the best according to the accuracy rate.
- 4. Based on Step 2, now also include bi-gram, and adjust min-df so that the number of columns is just below 200. then repeat step 3.
- 5. Based on step 4, now tighten the min-df by 1, then repeat step 3.

- Create a 3 by 4 table, table reporting the accuracy rates of the four models under the three types of representations


MELIORA